TwinOps

What if AI had to prove its fix before touching production?

TwinOps reproduces an incident in an isolated digital twin, tests competing fixes there, attacks its own best answer and hands a human only the change that survived the evidence.

`Design principle: don’t trust the AI’s reasoning. Make it prove the change.`

The problem: fixes that sound right

Under incident pressure, the first fix that restores service wins, whether or not it is safe.

- **Tempting fixes** such as disabling a filter, pinning hosts or turning off TLS validation make the symptom go away and open new risk.
- **Change review is re-investigation.** Approvers redo the diagnosis because there is no shared evidence.
- **AI makes it faster, not safer.** A fluent explanation is not evidence, and a model can be steered by text in logs.

REAL EXAMPLE FROM OUR LAB

Disabling the edge filter restores every customer journey *and* exposes the management console to an untrusted edge.

```
NEG-1 admin console 10.99.0.10:8443 answered HTTP from untrusted 10.23.0.1
```

The TwinOps loop

1 Observe

2 Diagnose

3 Reproduce in twin

4 Experiment

5 Challenge

6 Proof Gate

7 Human approval

8 Apply

ALLOWED TO BE WRONG SAFELY

Every candidate is applied to a twin of the affected slice and measured with real probes. Failed fixes cost nothing.

ATTACKS ITS OWN ANSWER

The challenge role tries IPv6 parity, a secondary node failure, warm caches and session continuity against the best fix.

PRODUCTION STAYS READ-ONLY

The only write path is an apply gateway that needs a human approval bound to the change hash and the target-state hash.

Live demo: one real run on the emulated lab

- Fault reproduced in the twin: 10/10 symptoms match
- A · Restart routing → REJECTED: J1, J3 still fail (TCP timeout, curl exit 28)
- B · Roll back prefix-list → IPv6 challenge fails → superseded
- B′ · Remove EDGE-IN seq 30 only → 7/7 automated gates pass
- U · Disable the filter (evaluation case) → REJECTED: admin console exposed
- Human approves the exact hash → applied → 11 real post-checks pass

58.5 s
server compute to verdict (FRR, CoreDNS, nginx in Containerlab)

0
production writes during diagnosis

2
unsafe or insufficient fixes rejected with evidence

Architecture

Web workbench · React

SSE event stream, evidence tabs, approval dialog

⇄

FastAPI on Cloud Run

incidents · SSE · decisions · Proof Pack export · CSP, rate limit, operator auth

Planner

Gemini (structured output) with deterministic runbook fallback

Twin backend

model twin (Cloud Run) · Containerlab lab: FRR BGP, CoreDNS, nginx TLS

Proof Gate G1–G8

deterministic code: reproduction, restore, regression, challenges, policy, rollback, latency, human

Apply gateway

single write path · approval token bound to change + state hash · post-checks · auto-rollback

Same event and evidence model in the browser engine, the Python engine and the lab, checked by 40 parity cases.

Where Gemini is used, and why code does the rest

GEMINI DOES THE REASONING

- Ranked hypotheses that cite evidence IDs (JSON schema)
- Fix proposals from a typed action catalogue
- Extra challenge ideas against its own best fix
- Approver briefing, labelled non-authoritative

CODE DOES THE JUDGING

- Twin, probes, gates and policy are deterministic
- Every model output is schema-validated and evidence-checked
- Device logs are fenced as untrusted data
- Call budget, then runbook fallback; record and replay for audits

Evidence: evaluation harness

Emulated lab, one run per fault (laptop, Docker Desktop)

FAULT	VERDICT	COMPUTE
F1 · Route misconfiguration B superseded by B′ after a real IPv6 timeout	PASS UNDER DEFINED TESTS	58.5 s
F2 · DNS failure C superseded by C′: warm resolver cache still stale	PASS UNDER DEFINED TESTS	41.1 s
F3 · Certificate expiry reload drains long-lived sessions (G7 warning)	PASS WITH WARNINGS	34.4 s
F0 · Link flap (unsupported) not reproducible in the twin, so no fix proposed	INCONCLUSIVE	18.3 s

40
seeded incidents (model twin)

0
unsafe fixes passed

6/6
adverse controls held

- Target drifts after the snapshot → approval invalidated
- Approval hash ≠ tested change → refused
- Operator rejects → zero writes
- Planner emits an off-catalogue action → never executed
- Unsupported physical fault → INCONCLUSIVE, zero experiments
- Instruction-like text in device logs → treated as data

Diagnosis accuracy shown here is the runbook baseline. The Gemini planner runs through the same harness: `python -m twinops.evaluation --planner llm`

Impact on enterprise operations work

ON-CALL ENGINEER

Starts from a reproduced fault and tested candidates instead of a blank terminal. The tempting fix is rejected before anyone tries it.

CHANGE APPROVER

Reviews a Proof Pack covering evidence, diff, rollback rehearsal and untested conditions, then approves a hash rather than a story.

ORGANISATION

Every incident leaves a replayable evidence bundle, so knowledge stops living in one person's head.

Pilot metrics to measure (no invented baselines): time from alarm to verified fix · change failure rate · approval review time · share of AI proposals rejected by the gate.

Scalability: packs and twin backends

- **Scenario packs**: regional telecom core and hybrid enterprise today. The topology, journeys and fault catalogue are data.
- **Twin backends** behind one protocol: an in-process model twin, then a Containerlab lab with real routing, DNS and TLS, then Kubernetes (kind), cloud-config sandboxes and CI/CD.
- **Planner providers**: Gemini API, Vertex AI, any OpenAI-compatible endpoint, or replay. The planner can change without touching the Proof Gate.

HONEST LIMITS TODAY

- Incident state is in memory, so the service runs as one instance; multi-instance needs a shared store.
- Surge and long-lived session behaviour are listed as untested in every Proof Pack.
- The lab is synthetic. Production would start in read-only shadow mode.

Roadmap and ask

1. Incident memory: past fixes are retested on today's twin, never reused blindly
2. Policy as code (OPA) and real telemetry (Prometheus, OpenTelemetry)
3. Read-only shadow mode next to a real NOC: verdicts recorded, nothing applied

Ask: one operations team, an approved LLM endpoint and a lab slice for a four-week shadow pilot.

> "The AI isn't impressive because it knew the answer. It's impressive because it was allowed to be wrong safely until it could prove one."